

GRADUATION PROJECT PROPOSAL

CardShield

Real-Time Card Transaction Guard

*A Real-Time Distributed Fraud Detection System Built on Apache Kafka, Apache Flink, Apache Airflow,
Hadoop HDFS and Apache HBase*

Prepared by: [Abdelrahman El-Tayeb]

Department: [Data Science], Faculty of Computer and Information Sciences

July 2026

Domain: FinTech · Fraud Detection · Real-Time Streaming Analytics

1. Introduction

1.1 Problem Statement

Credit card theft causes millions of dollars in financial losses every year, and the vast majority of conventional fraud detection systems identify fraudulent activity only after a transaction has already been completed and funds have been moved. This reactive posture leaves financial institutions and cardholders exposed to irreversible losses, chargebacks, and reputational damage. CardShield addresses this gap by intercepting suspicious activity before payment approval, continuously monitoring the live transaction stream and applying real-time security rules at the point of authorization rather than during post-transaction reconciliation.

The proposed architecture is built to deliver real-time transaction monitoring, behavioral analysis, geographic anomaly detection, fast blacklist propagation, reliable orchestration, distributed storage, fault tolerance, and horizontally scalable processing — all operating together as a single, cohesive pipeline capable of protecting millions of concurrent card transactions.

1.2 Project Objective

The objective of this project is to develop a real-time, distributed fraud detection system capable of identifying suspicious credit card transactions before payment completion. The system is built on Apache Kafka, Apache Flink, Hadoop HDFS, Apache HBase, and Apache Airflow, and is designed to process incoming transactions instantly, apply fraud detection rules, archive historical records for compliance, maintain up-to-date blacklist information, and orchestrate the entire pipeline automatically with minimal human intervention.

1.3 Project Scope

CardShield operates within the following domains:

- Financial Technology (FinTech) •
- Fraud Detection and Risk Scoring •
- Real-Time Data Processing •
- Streaming Analytics and Distributed Systems •

2. Dataset

Dataset Name: Credit Card Fraud Detection Dataset

Source: Kaggle

Data Type: High-dimensional numerical CSV dataset

The dataset exhibits the following characteristics that make it suitable for this project:

- Highly imbalanced class distribution between fraudulent and legitimate transactions
- Purely numerical, anonymized transaction features
- A binary fraud label for supervised evaluation
- A large volume of transaction records, enabling realistic load testing
- A structure well suited to simulate a continuous, real-time streaming feed

3. Overall System Workflow

The end-to-end pipeline moves data through a sequence of ingestion, orchestration, storage, processing, and decision stages. At a high level, the workflow proceeds as follows:

- Kaggle credit card transaction dataset (raw CSV files)
- Containerized ingestion environment (Docker)
- Apache Kafka — real-time transaction ingestion
- Apache Airflow — pipeline orchestration and control
- Hadoop HDFS — distributed audit-log archival
- Apache Flink — real-time fraud rules engine
- Apache HBase — real-time blacklist and fraud database
- Processed alerts and real-time fraud decision
- Power BI dashboard — monitoring and visualization

Apache Airflow sits alongside the streaming path as a control-plane component: it does not process transaction payloads directly but continuously supervises Kafka connectivity, triggers and monitors Flink jobs, and schedules periodic HBase backups into HDFS, ensuring the pipeline remains healthy, auditable, and self-recovering.

4. System Architecture

The architecture is organized into four cooperating layers: an ingestion layer (Kaggle dataset, CSV staging, and Docker-based ingestion), a control and orchestration layer (Apache Airflow), a real-time processing layer (Apache Kafka and Apache Flink), and a persistence layer (Hadoop HDFS for historical audit logs and Apache HBase for millisecond-latency lookups). Together these layers form a closed loop in which every transaction is ingested, screened, archived, and either approved or flagged within milliseconds.

CardShield: Real-Time Credit Card Fraud Detection Pipeline

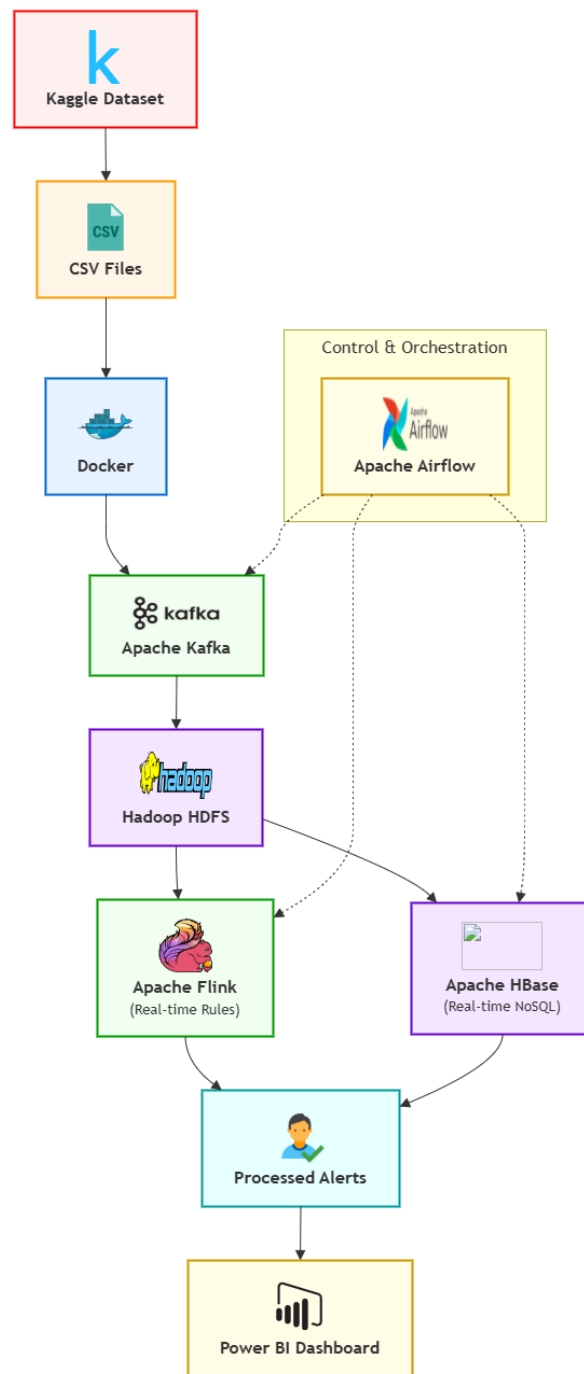


Figure 1: Real-Time Credit Card Fraud Protection Workflow Architecture

4.1 Component Connections

The historical dataset is retrieved from Kaggle and staged as CSV files, which are then loaded through a Docker-containerized ingestion process to guarantee a reproducible, isolated runtime environment. The ingestion process publishes transaction events onto Apache Kafka, which acts as the durable, high-throughput backbone of the streaming pipeline. Apache Airflow supervises this path through control signals — represented in the architecture as dotted orchestration lines — reaching into Kafka, Flink,

and HBase to verify health, trigger jobs, and schedule maintenance without sitting directly in the transaction data path.

From Kafka, transaction events flow into Hadoop HDFS for durable, replicated audit logging, and simultaneously into Apache Flink, which executes the real-time rules engine against each event. Flink consults and updates Apache HBase for millisecond-latency blacklist and profile lookups, and both Flink and HBase feed their outputs into a processed-alerts stage. Finally, processed alerts and fraud decisions are surfaced on a Power BI dashboard, giving fraud analysts and operations teams real-time visibility into system activity and flagged transactions.

5. Workflow Explanation

5.1 Kaggle Dataset and CSV Staging

The Kaggle Credit Card Fraud Detection dataset provides the historical transaction data used to simulate a live payment feed. It exists to give the pipeline a realistic, labeled, and imbalanced dataset that mirrors real-world fraud rates. It was selected because it is a well-established, publicly available benchmark for fraud detection research, and its numerical, anonymized features make it directly usable without additional data-cleansing overhead. The dataset's output is a set of structured CSV files that feed into the containerized ingestion stage.

5.2 Docker Containerization

Docker packages the ingestion tooling and its dependencies into a portable, reproducible runtime, solving the problem of environment drift between development and production. It takes the staged CSV files as input and produces a consistent transaction event stream as output, which is then published to Apache Kafka, ensuring that every environment running the pipeline behaves identically.

5.3 Apache Kafka — Real-Time Ingestion

Kafka exists to decouple transaction producers from downstream consumers and to guarantee that no transaction event is lost even under heavy load. It was selected for its proven horizontal scalability, high throughput, and fault-tolerant, replicated log design. Kafka's input is the transaction event stream from the ingestion layer, and its output is a durable, ordered stream of events consumed simultaneously by HDFS and Flink, interacting with Airflow for health supervision.

5.4 Apache Airflow — Orchestration

Airflow exists to coordinate and monitor every moving part of the pipeline so that no manual intervention is required for routine operations. It was chosen for its mature DAG-based scheduling model, its rich monitoring and alerting ecosystem, and its ability to safely recover from partial failures. Airflow's inputs are the health signals and job states of Kafka, Flink, and HBase, and its output is a continuously supervised, self-healing pipeline that can enter a protective Safe Mode when anomalies are detected.

5.5 Hadoop HDFS — Distributed Audit Archive

HDFS exists to provide durable, replicated, long-term storage for every transaction event, satisfying regulatory and audit requirements that real-time systems alone cannot meet. It was selected for its proven fault tolerance, block-level replication, and ability to scale linearly with data volume. HDFS receives the full transaction stream from Kafka as input and produces a complete, queryable historical archive as output, which also underpins Airflow's periodic HBase backup routine.

5.6 Apache Flink — Real-Time Rules Engine

Flink exists to apply fraud detection logic to every transaction with sub-second latency, which is the central requirement of the project. It was chosen for its native support for event-time processing, stateful sliding-window computation, and exactly-once processing guarantees, all of which are essential for accurate velocity and behavioral checks. Flink consumes transaction events from Kafka and blacklist and profile data from HBase as input, and produces fraud scores, alerts, and decisions as output, which are written back to HBase and forwarded to the alerts stage.

5.7 Apache HBase — Real-Time Fraud Database

HBase exists to provide millisecond-latency lookups for blacklisted cards, credit limits, and recent transaction history — data that Flink must consult on every single event. It was selected for its column-family NoSQL design, which scales horizontally while preserving fast random access. HBase's input is the stream of fraud scores and profile updates produced by Flink and Airflow's backup routine, and its output is the authoritative, continuously updated Fraud Guard Database consulted in real time.

5.8 Processed Alerts and Power BI Dashboard

The processed-alerts stage consolidates the outputs of Flink and HBase into a unified decision feed, which is then visualized through a Power BI dashboard. This final stage exists to give human analysts a clear, real-time operational view of system activity, flagged transactions, and pipeline health, closing the loop between automated detection and human oversight.

6. Technology Stack

Technology	Purpose	Why Used
Apache Kafka	Real-time transaction ingestion and event streaming	High throughput, durability, and horizontal scalability
Apache Airflow	Pipeline orchestration, scheduling, and monitoring	Mature DAG scheduling with robust failure recovery
Apache Flink	Real-time fraud rules engine and stream processing	Low-latency, stateful, exactly-once stream processing

Hadoop HDFS	Distributed storage of historical audit logs	Fault-tolerant, replicated, linearly scalable storage
Apache HBase	Real-time blacklist and fraud lookup database	Millisecond-latency NoSQL access at scale
Kaggle Dataset	Source of realistic historical transaction data	Publicly available, labeled, and imbalanced benchmark data

Apache Kafka forms the ingestion backbone of the system, buffering and distributing transaction events with guaranteed ordering and durability. Apache Airflow provides the control plane, coordinating scheduling and recovery across every other component without touching the transaction data directly. Apache Flink executes the core fraud detection logic in real time, leveraging windowed state to reason about transaction velocity and behavior. Hadoop HDFS preserves a complete, compliant historical record of every event, while Apache HBase supplies the sub-millisecond lookups required for blacklist and profile checks. The Kaggle dataset underlies the entire simulation, standing in for a live payment gateway feed during development and testing.

7. Detailed Component Descriptions

7.1 Apache Kafka

Purpose: Serves as the durable, high-throughput messaging backbone that ingests every incoming card transaction as an ordered event stream.

Responsibilities:

- Receiving transaction events from the ingestion layer
- Partitioning and replicating topics for scalability and durability
- Delivering events simultaneously to HDFS and Flink consumers

Advantages:

- Extremely high throughput and horizontal scalability
- Strong durability guarantees through replicated logs
- Wide ecosystem support and mature tooling

Limitations:

- Operational complexity in cluster management
- Requires careful partition and topic design for optimal performance

Integration with the System: Acts as the single entry point for all transaction data, feeding both the archival path (HDFS) and the real-time detection path (Flink), while Airflow monitors its health.

Best Practices: Use adequate topic partitioning aligned to expected throughput; enable replication for fault tolerance; monitor consumer lag continuously.

Expected Output: A continuously healthy, low-latency stream of transaction events available to all downstream consumers.

7.2 Apache Airflow

Purpose: Orchestrates, schedules, and monitors every operational task in the fraud detection pipeline, ensuring the system runs autonomously and recovers safely from failures.

Responsibilities:

- Verifying Kafka connectivity and stream health
- Triggering and supervising Apache Flink fraud detection jobs
- Scheduling recurring HBase backups into HDFS

Advantages:

- Clear, auditable DAG-based workflow definitions
- Built-in retry policies and alerting
- Extensible integration with virtually any external system

Limitations:

- Not designed for sub-second data processing itself
- Scheduler can become a bottleneck if misconfigured for very high task volumes

Integration with the System: Sits alongside — not inside — the transaction data path, issuing control and monitoring signals to Kafka, Flink, and HBase, and entering Safe Mode if Flink fails.

Best Practices: Set conservative but sufficient retry counts and delays; centralize logging; define explicit SLAs per task; separate orchestration logic from business logic.

Expected Output: A fully supervised pipeline with automatic recovery, scheduled backups, and clear audit trails of every operational event.

7.3 Apache Flink

Purpose: Executes the real-time fraud detection rule engine, scoring every transaction within milliseconds of its arrival.

Responsibilities:

- Applying geographic, velocity, and behavioral fraud rules
- Maintaining stateful sliding windows over transaction history
- Producing fraud scores and triggering alerts

Advantages:

- True low-latency, event-time stream processing
- Exactly-once processing semantics for consistent state
- Native support for complex windowed computations

Limitations:

- Higher operational complexity than simpler batch frameworks
- State backend sizing must be carefully tuned for large key spaces

Integration with the System: Consumes transaction events from Kafka and profile/blacklist data from HBase, writing fraud decisions back to HBase and forwarding alerts downstream.

Best Practices: Use incremental checkpointing for large state; tune window sizes to balance latency and accuracy; isolate rule logic into modular, testable operators.

Expected Output: Real-time fraud scores and flags for every transaction, with sub-second end-to-end latency.

7.4 Hadoop HDFS

Purpose: Provides distributed, replicated, long-term storage for the complete history of transaction events, supporting compliance and audit requirements.

Responsibilities:

- Storing raw and processed transaction logs durably
- Replicating data blocks across nodes for fault tolerance
- Serving as the destination for scheduled HBase backups

Advantages:

- Proven fault tolerance through block replication
- Scales linearly with data volume
- Well-suited for large sequential read/write workloads

Limitations:

- Higher latency than in-memory or NoSQL stores for random access
- Not intended for real-time transactional queries

Integration with the System: Receives the full event stream from Kafka for archival and receives periodic HBase table backups orchestrated by Airflow.

Best Practices: Configure an appropriate replication factor; separate hot and cold storage tiers; monitor NameNode health closely.

Expected Output: A complete, durable, and auditable historical record of all transaction activity.

7.5 Apache HBase

Purpose: Delivers millisecond-latency lookups for blacklisted cards, credit limits, and recent transaction history that Flink must check on every event.

Responsibilities:

- Storing and serving the real-time blacklist database

- Maintaining up-to-date credit limit and recent-transaction records
- Providing fast random-access reads and writes to Flink

Advantages:

- Millisecond-level read/write latency at scale
- Horizontally scalable column-family storage model
- Strong consistency for single-row operations

Limitations:

- Operationally complex to tune and administer
- Less efficient for large analytical scans than HDFS

Integration with the System: Serves as the authoritative real-time data source consulted by Flink during rule evaluation, and is periodically backed up into HDFS by Airflow.

Best Practices: Design row keys to avoid hotspotting; pre-split regions for expected load; monitor compaction cycles closely.

Expected Output: A continuously updated, low-latency Fraud Guard Database supporting real-time decisioning.

8. Apache Airflow Orchestration in Depth

8.1 Why Airflow?

Airflow is responsible for orchestrating and monitoring the complete fraud detection pipeline. It schedules maintenance operations, verifies Kafka connectivity, supervises Flink jobs, performs HBase backups, and ensures secure, autonomous pipeline execution around the clock.

8.2 Airflow DAG

DAG Name: Fraud Protection Pipeline DAG

Execution order:

- test_card_gateway_stream
- run_rules_guard_job
- hbase_table_backup

8.3 DAG Tasks

test_card_gateway_stream

Verifies that Kafka continuously receives incoming payment transactions.

Expected Output: Confirmation that the Kafka stream is healthy.

run_rules_guard_job

Triggers and monitors the Apache Flink fraud detection jobs. Responsibilities include:

- Applying geographic rules •
- Behavioral analysis •
- Velocity detection •
- Fraud scoring •
- Triggering alerts •

Expected Output: Suspicious transactions are identified and scored.

hbase_table_backup

Compresses and backs up HBase tables into HDFS.

Purpose:

- Security •
- Auditing •
- Regulatory compliance •
- Disaster recovery •

Expected Output: A secure, compressed historical backup of the fraud database.

8.4 Scheduling

Frequency: Runs daily

Scheduled Time: 12:30 AM

8.5 Retry Policy

Retries: 4

Retry Delay: 5 minutes

8.6 Monitoring

The pipeline is monitored through centralized logging that aggregates task output across all DAG runs, continuous pipeline and health checks for each component, real-time performance metrics covering throughput and latency, and automated alert generation whenever thresholds are breached.

8.7 Failure Handling — Safe Mode

If Apache Flink fails, Airflow immediately stops automatic transaction approval and enables a protective Safe Mode. In this mode, the security team is notified, an alert is generated, customer funds are protected from unreviewed approvals, and the incident is fully documented for post-mortem analysis, ensuring that a processing failure never translates into unchecked financial risk.

9. Fraud Detection Rules

The rules engine applies a combination of geographic, behavioral, and threshold-based checks to every transaction. The table below summarizes the primary detection rules.

Rule	Description
Geographic velocity check	Transactions occurring in different cities within a five-minute window are flagged as physically implausible.
High-value burst detection	Multiple high-value purchases occurring in a short period are treated as a strong fraud indicator.
Behavioral anomaly detection	Sudden deviations from a cardholder's established spending pattern are scored as suspicious.
Credit limit enforcement	Transactions that exceed the cardholder's approved credit limit are automatically blocked.
Unusual country check	Transactions originating from countries atypical for the cardholder trigger elevated scrutiny.
Blacklist enforcement	Transactions involving blacklisted cards are rejected immediately upon lookup in HBase.
Velocity checks	The overall transaction frequency per card is monitored to catch rapid, automated abuse patterns.
Behavioral profiling	Long-term spending profiles are maintained to distinguish genuine anomalies from natural variation.

10. Functional Requirements

- The system shall ingest credit card transaction events in real time via Apache Kafka.
- The system shall apply fraud detection rules to every transaction before payment approval.
- The system shall maintain and consult a real-time blacklist of known fraudulent cards.
- The system shall archive all transaction events durably for audit and compliance purposes.
- The system shall generate real-time alerts for transactions flagged as suspicious.
- The system shall orchestrate all scheduled maintenance and monitoring tasks automatically.
- The system shall provide a dashboard for visualizing fraud alerts and pipeline health.
- The system shall support automated backup and recovery of the fraud database.

11. Non-Functional Requirements

- Availability:** The pipeline shall remain operational continuously, with automated recovery from component failures.
- Scalability:** The system shall scale horizontally to accommodate growing transaction volumes.
- Reliability:** The pipeline shall deliver consistent, accurate fraud decisions under sustained load.
- Performance:** The system shall process and score transactions within sub-second latency.
- Fault Tolerance:** Component failures shall not result in data loss or unchecked transaction approval.
- Maintainability:** The codebase and DAGs shall be modular and easy to extend or modify.
- Security:** All data in transit and at rest shall be protected against unauthorized access.
- Latency:** End-to-end transaction scoring shall complete before payment authorization.
- Monitoring:** All components shall expose health and performance metrics for continuous observability.
- Recoverability:** The system shall be able to restore state quickly from HDFS-backed HBase backups.

12. Expected Outputs

- Kaggle Dataset / CSV Staging — a structured, streaming-ready transaction feed.
 - Docker Ingestion — a consistent, reproducible transaction event stream.
 - Apache Kafka — a durable, ordered stream of transaction events.
 - Apache Airflow — a fully supervised, self-recovering pipeline with audit logs.
 - Hadoop HDFS — a complete, replicated historical archive of all transactions.
 - Apache Flink — real-time fraud scores and alerts for every transaction.
 - Processed Alerts / Power BI — a live operational view of flagged activity.
- The final and overarching output of the system is an Active Fraud Guard Database stored inside HBase, continuously updated and consulted in real time to protect every incoming transaction.

13. Deliverables

- Fraud Detection Pipeline (end-to-end implementation)
- Apache Airflow DAG (Fraud Protection Pipeline DAG)
- Apache Kafka Topics and Configuration
- Apache Flink Jobs (Rules Engine)
- Apache HBase Tables (Fraud Guard Database)
- Hadoop HDFS Archive
- System Architecture Diagram
- Deployment Guide

- Technical Documentation •
- Final Project Report •

14. Implementation Challenges

- False positives:** Overly aggressive rules risk blocking legitimate transactions, requiring careful threshold tuning.
- Streaming latency:** Maintaining sub-second scoring under peak load demands efficient Flink job design.
- Cluster failures:** Node or service outages must be absorbed without disrupting the fraud decision path.
- Kafka delays:** Backpressure or broker slowdowns can propagate latency through the entire pipeline.
- State synchronization:** Keeping Flink's in-memory state consistent with HBase requires careful design.
- Large-scale processing:** Sustaining throughput as transaction volume grows requires horizontal scaling discipline.
- Data consistency:** Ensuring HDFS archives and HBase live data remain reconciled after backups and restores.
- Security:** Protecting sensitive financial data across every layer of the pipeline demands strict access controls.

15. Future Improvements

- Incorporating machine learning-based fraud scoring alongside rule-based detection •
- Graph-based fraud detection to uncover collusive or networked fraud patterns •
- Expanded real-time dashboards with predictive analytics •
- Migration to a cloud-native deployment model •
- Containerization of all pipeline components for portability •
- Kubernetes-based orchestration for elastic scaling •
- CI/CD pipelines for automated testing and deployment •
- Continuous model retraining to adapt to evolving fraud patterns •

16. Conclusion

CardShield presents a comprehensive, distributed approach to real-time credit card fraud detection, combining the strengths of Apache Kafka, Apache Flink, Apache Airflow, Hadoop HDFS, and Apache HBase into a single, cohesive pipeline. By intercepting suspicious transactions before payment

approval rather than after the fact, the system directly addresses the core weakness of traditional, reactive fraud detection approaches. Its layered architecture — spanning ingestion, orchestration, real-time processing, and durable storage — ensures that the pipeline remains scalable, fault-tolerant, and auditable as transaction volumes grow. With clearly defined functional and non-functional requirements, a well-structured Airflow DAG, and a concrete set of fraud detection rules, CardShield provides a solid foundation for a production-grade, real-time financial protection system, while leaving clear pathways for future enhancement through machine learning, graph analytics, and cloud-native deployment.